

## AU324 - PROJET DRONE

MODÉLISATION ET ANALYSE DYNAMIQUE DES AÉRONEFS

# Modélisation et simulation du comportement dynamique d'un quadrimoteur

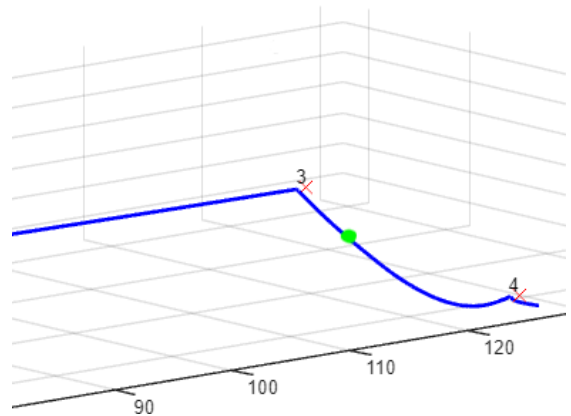


FIGURE 1 : Changement de position commandée du quadrimoteur.

BUSOLINI Hugo  
GAUTRON Pierre  
GRANDPERRET Alexi

## PRÉAMBULE

*Ce manuscrit s'inscrit dans le cadre de l'acquisition envisagée par M. JACOB Adrien d'un drone quadrirotor de notre firme. Comme première étape de cette commande, ce document présente notre simulation numérique du comportement de l'aéronef à travers l'environnement Matlab-Simulink.*

*Nous proposons ainsi une modélisation numérique du drone, de sa dynamique de réponse à un pilotage ainsi que de son intégration à l'environnement. Ces travaux s'inscrivent donc dans une démarche de validation du projet de drone soumis au cahier des charges imposé avant tout potentiel prototypage physique.*

## Table des matières

|            |                                                                       |           |
|------------|-----------------------------------------------------------------------|-----------|
| <b>I</b>   | <b>FONDAMENTAUX</b>                                                   | <b>2</b>  |
| I-1        | Paramétrage et mécanique de vol . . . . .                             | 2         |
| I-2        | Compléments sur la dynamique du drone . . . . .                       | 3         |
| I-3        | Hypothèses admises . . . . .                                          | 4         |
| <b>II</b>  | <b>MODÉLISATION &amp; SIMULATION</b>                                  | <b>7</b>  |
| II-1       | Modélisation numérique du système (TP_01) . . . . .                   | 7         |
| II-1.A     | Fondamentaux de la représentation d'état . . . . .                    | 7         |
| II-1.B     | Détermination des matrices . . . . .                                  | 8         |
| II-2       | Bases de la simulation - Contrôle simple et hover . . . . .           | 9         |
| II-2.A     | Compensation de l'environnement . . . . .                             | 9         |
| II-2.B     | Hover du drone . . . . .                                              | 9         |
| II-2.C     | Rotation constante en lacet . . . . .                                 | 9         |
| II-2.D     | Changement de repère . . . . .                                        | 10        |
| II-2.E     | Sous-conclusion . . . . .                                             | 10        |
| II-3       | Corps de la simulation - Commande et asservissement (TP_02) . . . . . | 12        |
| II-3.A     | Calculateur des commandes moteurs . . . . .                           | 12        |
| II-3.B     | Asservissement du système . . . . .                                   | 14        |
| II-3.C     | Sous-conclusion . . . . .                                             | 16        |
| II-4       | Annexe . . . . .                                                      | 16        |
| II-5       | Propositions supplémentaires . . . . .                                | 17        |
| II-5.A     | Suivi de position . . . . .                                           | 17        |
| II-5.B     | Parcours urbain . . . . .                                             | 18        |
| <b>III</b> | <b>OUVERTURE</b>                                                      | <b>20</b> |

Les présentations des études TP\_01 et TP\_02 ont été limitées aux 5 pages imposées. En revanche, en l'absence de consigne pour la taille du rapport et d'une volonté du groupe, nous n'avons pas appliqué de telles limites aux parties **FONDAMENTAUX** ou **CONCLUSION**. Nous présenterons ainsi ces sections avec la précision et le détail que nous jugeons essentiels à la bonne compréhension du livrable pour M. JACOB Adrien, client du quadricoptère.

# I FONDAMENTAUX

## I-1 Paramétrage et mécanique de vol

Pour modéliser ce projet au mieux, nous devons tout d'abord poser les bases physiques avant pouvoir les manipuler correctement. Voici certaines informations qui seront clés pour assurer la compréhensibilité du projet.

➤ L'étude se fera sur **un quadrirotor en configuration « X »** évoluant dans un environnement dont les caractéristiques sont définies pour avoir une certaine **simplification**. Ce drone est donc modélisé dans un cadre tridimensionnel avec un seul facteur environnemental pris en compte : **la gravité**. Elle est **constante**, **uniforme** et sera exprimée dans le repère terrestre  $R_t$ . Les effets aérodynamiques comme la **traînée** ou le **vent** sont **volontairement négligés** pour **faciliter l'étude** de ce projet.

Il y a donc deux repères sur lesquels nous allons travailler :

- **Le repère terrestre**  $R_t (x_t, y_t, z_t, \varphi_t, \theta_t, \psi_t)$ . C'est un repère fixe dans lequel sont exprimées les positions, trajectoires et la gravité agissant sur le drone.
- **Le repère du drone**  $R_B (x_B, y_B, z_B, \varphi_B, \theta_B, \psi_B)$ . Ce repère est mobile, est solidaire au châssis du drone et il est placé sur son centre de gravité. Les vitesses angulaires et couples seront donc exprimés dans ce repère.

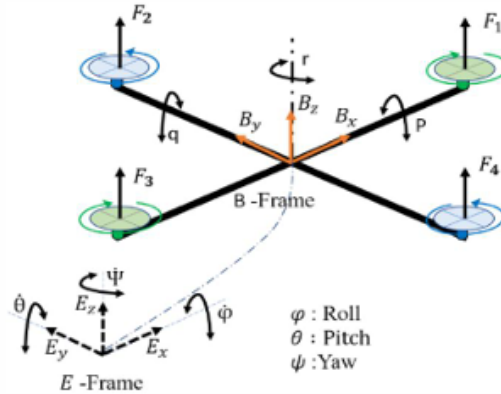


FIGURE 2 : Schéma d'un drone quadrirotor en configuration « X » avec les axes du repère lié au drone, les forces moteurs et les angles d'Euler  $(\varphi, \theta, \psi)$ .

Le changement de repère entre  $R_b$  et  $R_t$  s'effectuera par des **matrices de passage d'Euler** dans l'ordre **roll-pitch-yaw**  $(\varphi, \theta, \psi)$ .

➤ Chaque moteur génère une force de poussée verticale  $F_{Mi}$  et un couple de rotation  $C_{Mi}$ . Ils vont être caractérisés par deux coefficients :  $k_t$  le coefficient de poussée, et  $k_c$  le coefficient de couple.

Nous définissons donc :

$$k_t = 0,8 \quad \text{et} \quad k_c = 0,2$$

Cela implique que 80% de portance et 20% de couple induit (action-réaction) pour chaque moteur.

## I-2 Compléments sur la dynamique du drone

Dans le cadre du TP\_01, la rotation du drone en lacet est uniquement déclenchée par la définition de conditions initiales, telles qu'un angle ou une vitesse angulaire non nulle. Aucune commande moteur n'était réellement appliquée pour provoquer volontairement ces rotations : le drone évoluait librement, selon une dynamique initialement imposée, dans un environnement sans vent ni traînée.

Pour la suite de la simulation, nous souhaitons aller plus loin en permettant au drone de réaliser lui-même des rotations autour de ses axes — roll ( $\varphi$ ), pitch ( $\theta$ ) et yaw ( $\psi$ ) — en réponse directe à des **commandes de pilotage**. C'est donc à travers l'action combinée de ses quatre moteurs que le drone devra s'orienter activement dans l'espace, sans dépendre de conditions initiales préprogrammées.

Pour cela, nous rappelons les principes physiques qui gouvernent la mécanique de vol d'un quadrirotor. De haut en bas, gauche à droite, nous nommons les moteurs  $M_1$ ,  $M_2$ ,  $M_3$ ,  $M_4$

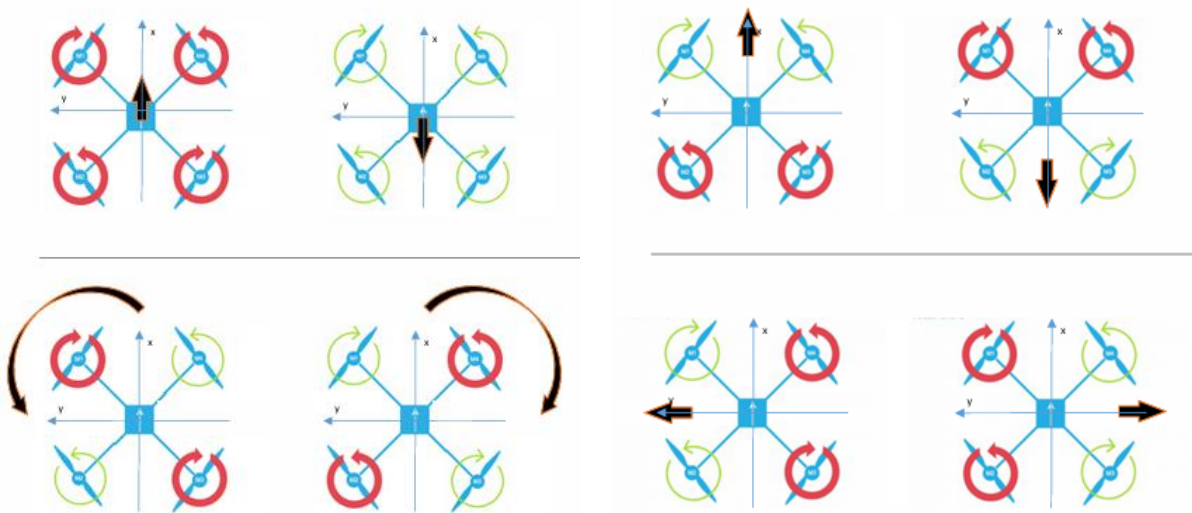


FIGURE 3 : Effets des commandes moteurs sur les mouvements du drone : rotation (gauche inférieure), montée et descente (gauche supérieure), translation (droite).

- Une **variation différentielle des vitesses moteurs** permet de générer un moment autour d'un axe spécifique. Par exemple, en augmentant la poussée des moteurs  $M_1$  et  $M_3$  tout en diminuant celle de  $M_2$  et  $M_4$ , on génère un moment de rotation autour de l'axe vertical du drone, provoquant un **lacet** ( $\psi$ ).
- Un **déséquilibre avant/arrière** (ex. :  $M_2$  et  $M_3$  plus puissants que  $M_1$  et  $M_4$ ) crée une rotation en pitch ( $\theta$ ), inclinant le drone vers l'avant.
- De même, une **asymétrie gauche/droite** (ex. :  $M_4$  et  $M_3$  plus puissants que  $M_1$  et  $M_2$ ) provoque une rotation en roll ( $\varphi$ ), inclinant le drone latéralement.

Ce mécanisme est à la base de la **translation contrôlée** : en inclinant le drone (via pitch ou roll), on induit un déplacement horizontal. Cela nous permet de piloter le drone d'une **pose A** vers une **pose B**, en générant une trajectoire volontaire.

Il est donc indispensable de maîtriser la génération de ces rotations par le calcul des commandes moteurs. C'est pourquoi, dans le TP2, nous construirons une architecture de commande en boucle fermée afin d'asservir notre système. Capable de convertir les erreurs de position en

angles d'inclinaison, puis en commandes moteurs grâce au *motor mixing* (calculateur), le drone devient ainsi capable de réagir dynamiquement à un objectif de position.

### I-3 Hypothèses admises

➤ Tout d'abord, pour simplifier les calculs, **nous ne prendrons en compte qu'unique-  
ment la masse du châssis et des systèmes embarqués (réunit en une seule masse),  
ainsi que celle des quatre moteurs :**

En revanche, **les masses des bras du drone ne sont pas pris en compte dans notre modélisation.** Cette hypothèse permet de simplifier les calculs liés à la position du centre de masse, ce qui facilite ensuite l'évaluation des moments et de la dynamique du système autour de ce centre.

➤ Afin de simplifier l'étude, nous allons considérer dans ce projet uniquement des **petits angles** pour  $(\varphi, \theta)$ .

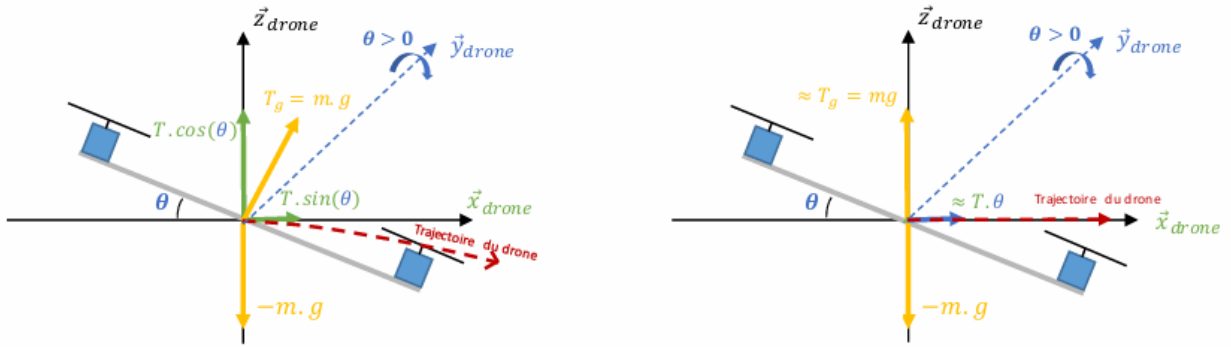


FIGURE 4 : Effet de l'angle de pitch  $\theta$  sur la trajectoire du drone : à gauche, sans approximation, le drone perd de l'altitude ; à droite, avec l'approximation des petits angles, la poussée verticale compense la gravité et permet de maintenir l'altitude.

Nous savons en effet que les **drones** fonctionnent de manière à ce que les **poussées** dans le repère du drone  $(x_B, y_B, z_B)$  soient déduites de la **poussée totale**  $T$  et des angles de **roll/pitch**  $(\varphi, \theta)$ .

Grâce à cette hypothèse, nous linéarisons le comportement de la poussée suivant  $\vec{x}_{\text{drone}}$  et  $\vec{y}_{\text{drone}}$ , fonction des angles  $(\varphi, \theta)$  :

$$\begin{pmatrix} T_x \\ T_y \\ T_z \end{pmatrix}_{R_{\text{drone}}} = \begin{pmatrix} T \cdot \sin \theta \\ -T \cdot \sin \varphi \\ \sqrt{(T \cdot \cos \theta)^2 + (T \cdot \cos \varphi)^2} \end{pmatrix}_{R_{\text{drone}}} \approx \begin{pmatrix} T \cdot \theta \\ -T \cdot \varphi \\ T \end{pmatrix}_{R_{\text{drone}}}$$

Cette **hypothèse** va permettre de **simplifier les calculs mathématiques**. Sans ces approximations, les équations ne sont plus **linéaires**, et, par conséquent plus complexes à résoudre.

Dans le cas pratique de notre projet, cela permettra de **négliger les pertes d'altitude** dues aux effets du roulis et du tangage, ce qui **facilite l'analyse du vol** et la **mise en place de commandes**.

Grâce à cela, on supposera que les **efforts produits par les moteurs** permettront de **compenser la gravité** indépendamment de l'**inclinaison du drone**.

- Après avoir posé ces hypothèses fondamentales, nous pouvons nous intéresser à la manière dont le système réponds dans son environnement -

➤ Pour cela, nous commençons par faire l'**hypothèse** que notre **signal moteur** est atteint instantanément par le système ( $\omega_{Mi} = \omega_{cMi}$ ). Ayant des **fournisseurs de renom** en qui nous avons pleinement confiance pour garantir une **commande rapide, robuste et précise** des moteurs, cette hypothèse est donc **pertinente**.

➤ Nous devons également considérer un **modèle de poussée** pour les moteurs. La littérature montre que la puissance transmise à l'air par une hélice est liée à la densité de l'air, à sa vitesse, et à la surface de l'hélice. Cette puissance étant proportionnelle au cube de la vitesse de rotation, on peut en déduire que la force générée par une hélice est proportionnelle au **carré de sa vitesse de rotation**.

Afin de **simplifier le problème**, nous adoptons une **loi proportionnelle** pour la modélisation des efforts générés par chaque moteur. La poussée verticale  $F_{Mi}$  générée par un moteur  $i$  est alors donnée par :

$$F_{Mi} = k_t \cdot \omega_{Mi}^2$$

et le moment de couple associé par :

$$\|\vec{M}_{Mi}(\vec{C}_{Mi})\| = \pm k_c \cdot \omega_{Mi}^2$$

Ainsi, la **poussée** et le **moment de la poussée** sont directement proportionnels au carré de la vitesse de rotation des hélices, ce qui nous permet d'exprimer plus simplement les forces et les couples agissant sur le drone dans la suite de l'étude.

- Enfin, pour assurer la cohérence du modèle, il est essentiel de considérer également le rôle des capteurs embarqués -

Un drone est un aéronef **criblé de capteurs** assurant son orientation et guidage. Pour cela, il embarque des capteurs comme l'*IMU*, le *baromètre*, le *GPS*, le *capteur à ultrasons*, la *caméra sol* et *FPV*, le *radar* et le *LiDAR*. Mais la complexité, inutile pour une première modélisation, de tous ces systèmes nous pousse donc à faire les hypothèses suivantes :

➤ Dans cette étude, nous allons donc **remplacer le système réel** drone par une **modélisation de son comportement dynamique**.

➤ En plus de cela, nous considérons que les capteurs du système réel sont capables de nous offrir des **informations sans défauts et instantanément**.

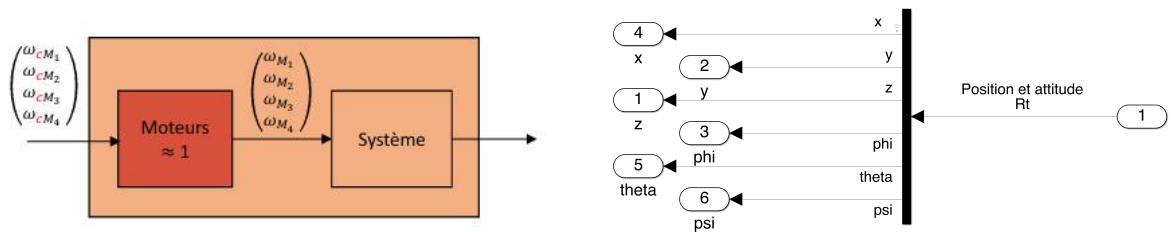


FIGURE 5 : Représentation des deux hypothèses précédentes. Gauche : Atteinte immédiate des consignes des moteurs. Droite : Sous-système Simulink - **Capteur**.

Cela revient à considérer un **retour unitaire** dans notre asservissement, avec une fonction de transfert capteur  $R_{\text{capteur}} = 1$ . Ce simple renvoi nous permettra donc de connaître :

— L'altitude du drone par rapport au sol :  $[z_{\text{drone}}, \dot{z}_{\text{drone}}]$  (ég : *ultrasons, baromètre*)

- Le placement horizontal du drone :  $[x_{\text{drone}}, y_{\text{drone}}]$  (ég : caméra sol, IMU)
- L'attitude angulaire du drone sur ses 3 axes (roll, pitch, yaw) :  $[\varphi_{\text{drone}}, \theta_{\text{drone}}, \psi_{\text{drone}}]$  (ég : IMU)

De ces capteurs, il est donc possible d'obtenir la position et la vitesse des trois translations et rotations du drone ainsi que les accélérations  $\ddot{x}_{\text{drone}}, \ddot{y}_{\text{drone}}$  dans le repère du drone  $R_{\text{drone}}$ .

Dans ce projet, nous commanderons uniquement les **positions** (*simplification*), ainsi nous récupérerons uniquement les **informations de position** pour nos **retours d'asservissement**.

Finalement, nous allons seulement commander le drone avec les **informations de position** et abandonner les **informations de vitesses/accélérations**. Cela va nous permettre, encore une fois, de **simplifier l'étude** et la **logique de contrôle** pour les besoins du **client**. Ce choix nous permet de rendre un travail plus **simple** et plus **facile à comprendre**, sans pour autant nuire aux **capacités finales** de notre drone.

---

## II MODÉLISATION & SIMULATION

Le contexte physique de la simulation étant à présent défini, rentrons désormais dans les premières étapes de la modélisation du drone.

### II-1 Modélisation numérique du système (TP\_01)

*L'objectif de cette première partie (TP1) consiste à **modéliser numériquement le drone** à partir de ses **principes physiques**, afin d'obtenir un **comportement fidèle à la réalité**. De plus, nous tâcherons de faire voler le drone à l'**équilibre (hover)** avant d'y appliquer une **rotation en lacet constante**.*

*Présentée en détail précédemment, cette **représentation d'état** constitue le **socle de notre simulation** : elle nous permet d'exprimer mathématiquement l'**évolution complète du drone** en fonction de ses **entrées moteurs** et de son **environnement**, tout en rendant le système **compatible avec les outils de simulation et de commande** développés par la suite.*

#### II-1.A Fondamentaux de la représentation d'état

Il nous faut modéliser le drone numériquement pour qu'il soit le plus proche du réel possible. Pour cela, il est d'usage d'utiliser une **représentation d'état** de la dynamique du système. Ce choix a été fait car il permet de décrire l'évolution des variables d'état du drone (positions, vitesses, angles, etc.) en fonction des entrées (ici les poussées des moteurs). Par ailleurs, grâce à l'hypothèse de **linéarisation** autour d'un point d'équilibre, cette forme reste suffisamment représentative du comportement réel du drone.

Nous commençons par définir l'équation d'état du système :

$$\dot{X} = AX + Bu$$

$x$  est ce que l'on appelle un **vecteur d'état**. Il importe que chacune des variables d'états apparaissent, de manière explicite ou implicite, sous forme dérivée dans le **jeu d'équations décrivant le système**. Ainsi, dans notre cas, nous définissons  $x$  comme étant :

$$X = [x \ y \ z \ \dot{x} \ \dot{y} \ \dot{z} \ \varphi \ \theta \ \psi \ \dot{\varphi} \ \dot{\theta} \ \dot{\psi}]^T$$

Nous définissons aussi le vecteur  $u$  comme étant le vecteur des **commandes du drone**. En l'occurrence, nous donnons à notre drone des vitesses de rotation de moteur qui, pour simplifier la modélisation, seront traduites par des **forces moteurs** auxquelles nous rajouterons l'influence de l'environnement dans lequel évolue le drone. Seule la **gravité** agit sur le drone. Ainsi, nous avons :

$$u = [F_{M_1} \ F_{M_2} \ F_{M_3} \ F_{M_4} \ -g]^T$$

Une fois l'équation d'état établie, nous devons définir l'équation de sortie :

$$Y = CX + Du$$

Il ne nous reste donc plus qu'à déterminer les matrices  $A$ ,  $B$ ,  $C$ ,  $D$  afin de compléter la représentation d'état.

## II-1.B Détermination des matrices

Pour cela, il nous faut utiliser le **théorème de la résultante dynamique** et du **moment dynamique** afin d'obtenir des équations différentielles du second degré selon les différentes variables d'états. Nous utiliserons par ailleurs le théorème du transport, appliqué aux moments.

Le **Théorème de la résultante dynamique** nous dit :

$$\begin{pmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{pmatrix}_{G \in S/R_T} = \frac{1}{m_{tot}} \cdot \sum \vec{F}_{ext} \quad G \in S/R_T$$

Pour faciliter la compréhension, nous allons prendre un exemple d'un de nos calculs des coefficients de ces matrices :

Pour le mouvement de 'hover', les valeurs de  $\ddot{x}$  et de  $\ddot{y}$  sont nulles, ainsi on a :

$$\ddot{z} = \frac{1}{m_{tot}} \sum \vec{F}_{ext} \quad G \in S/R_T \leftrightarrow \ddot{z} = \frac{F_{M1}}{m_{tot}} + \frac{F_{M2}}{m_{tot}} + \frac{F_{M3}}{m_{tot}} + \frac{F_{M4}}{m_{tot}} - g$$

Finalement, en appliquant l'égalité  $\dot{X} = AX + Bu$ , nous pouvons par exemple conclure que la sixième ligne de la matrice A est remplie de 0. En effet, aucune des variables d'états n'interviennent dans l'équation de Théorème de la résultante dynamique.

De la même manière nous pouvons conclure que la 6ème ligne de la matrice B est donc :

$$\begin{bmatrix} 1/m_{tot} & 1/m_{tot} & 1/m_{tot} & 1/m_{tot} & 1 \end{bmatrix}$$

Ce qui nous permet effectivement d'écrire à partir du système matriciel une des équations qui regit notre drone.

Nous appliquons également la même méthode pour le **Théorème du moment dynamique** que voici :

$$I_{G \in S/R_T} \cdot \begin{pmatrix} \ddot{\phi} \\ \ddot{\theta} \\ \ddot{\psi} \end{pmatrix}_{S/R_T} = \sum \vec{M}_{G \in S/R_T}(\vec{F}_{ext})$$

En épargnant le lecteur des autres calculs, nous obtenons les matrices suivantes :

```
% Matrice A d'état
A = [0 0 0 1 0 0 0 0 0 0 0 0;
      0 0 0 0 1 0 0 0 0 0 0 0;
      0 0 0 0 0 1 0 0 0 0 0 0;
      0 0 0 0 0 0 0 g 0 0 0 0;
      0 0 0 0 0 0 -g 0 0 0 0 0;
      0 0 0 0 0 0 0 0 0 0 0 0;
      0 0 0 0 0 0 0 0 0 1 0 0;
      0 0 0 0 0 0 0 0 0 0 1 0;
      0 0 0 0 0 0 0 0 0 0 0 1;
      0 0 0 0 0 0 0 0 0 0 0 0;
      0 0 0 0 0 0 0 0 0 0 0 0;
      0 0 0 0 0 0 0 0 0 0 0 0;
      0 0 0 0 0 0 0 0 0 0 0 0];
```

```
% Matrice B de commande
B = [0 0 0 0 0;
      0 0 0 0 0;
      0 0 0 0 0;
      0 0 0 0 0;
      0 0 0 0 0;
      1/m 1/m 1/m 1/m 1;
      0 0 0 0 0;
      0 0 0 0 0;
      0 0 0 0 0;
      0 0 0 0 0;
      L/Ix L/Ix -L/Ix -L/Ix 0;
      -L/Iy L/Iy L/Iy -L/Iy 0;
      kc/(Iz*kt) -kc/(Iz*kt) kc/(Iz*kt) -kc/(Iz*kt) 0];
```

```
% Matrice C d'observation
C = [1 0 0 0 0 0 0 0 0 0 0 0;
      0 1 0 0 0 0 0 0 0 0 0 0;
      0 0 1 0 0 0 0 0 0 0 0 0;
      0 0 0 0 0 0 1 0 0 0 0 0;
      0 0 0 0 0 0 0 1 0 0 0 0;
      0 0 0 0 0 0 0 0 1 0 0 0];
```

```
% Matrice D
D = [0 0 0 0 0;
      0 0 0 0 0;
      0 0 0 0 0;
      0 0 0 0 0;
      0 0 0 0 0;
      0 0 0 0 0;
      0 0 0 0 0];
```

FIGURE 6 : Représentation des matrices du système dynamique

## II-2 Bases de la simulation - Contrôle simple et hover

### II-2.A Compensation de l'environnement

Nous choisissons donc pour commencer de simuler notre drone de manière à ce que celui-ci compense exactement la gravité. Pour cela nous devons jouer sur le vecteur  $u$ .

Pour que le drone reste immobile alors que la gravité agit sur lui, les **4 moteurs combinés doivent donc parfaitement compenser le poids**. Il est donc évident que chacun des 4 moteurs doit compenser 1/4 du poids. Ainsi nous avons  $u$  défini comme :

$$u = \begin{bmatrix} \frac{mg}{4} & \frac{mg}{4} & \frac{mg}{4} & \frac{mg}{4} & -g \end{bmatrix}$$

Implémenté dans notre environnement Simulink, nous obtenons alors l'architecture logique de représentation d'état suivante :

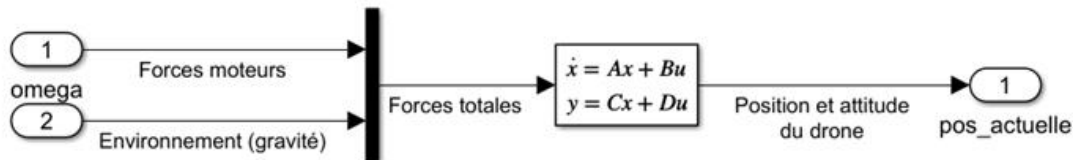


FIGURE 7 : Sous-système Simulink - **Système de commande** (TP\_01)

### II-2.B Hover du drone

Avant de pouvoir commander à une position souhaitée l'aéronef ou le faire évoluer dans un environnement, il est indispensable de comprendre et formaliser sa dynamique de base. Il est donc logique de commencer par une tâche plus simple à réaliser pour le drone.

Afin d'obtenir une simulation en Hover du drone, il nous suffit simplement de **compenser la gravité**. Il n'y a donc qu'à utiliser le vecteur  $u$  défini lors de la section précédente et un vecteur d'état initial avec des **valeurs de  $\dot{x}$ ,  $\dot{y}$ ,  $\dot{z}$ ,  $\dot{\phi}$ ,  $\dot{\theta}$  et  $\dot{\psi}$  nulles**

### II-2.C Rotation constante en lacet

À cet instant de la simulation, nous ne nous sommes pas encore préoccupés des **3 axes de rotation** du drone. Nous ne sommes donc pas encore en capacité de **contrôler ces rotations**. Simplement, nous souhaitons vérifier qu'une **rotation en lacet n'influe pas sur la stabilité de notre aéronef**.

Même si nous ne pouvons pas encore agir directement sur les rotations, nous avons la main sur **les conditions initiales** associées.

Nous imposons donc dans notre **vecteur d'état** des valeurs aux variables  $\psi$  et  $\dot{\psi}$ . Plus précisément, nous fixons :  $\psi = 0$  et  $\dot{\psi} = 20$  deg/s.

De cette manière, nous **imposons une rotation** sur notre drone au départ de la simulation. Compte tenu du fait que les actions de l'air ne sont pas prises en compte, le drone continuera de tourner par **l'effet de l'inertie**.

### II-2.D Changement de repère

Enfin, il semble important de préciser que nous avons, tout au long de cette partie, fait l'**hypothèse** que les **référentiels terrestre** et celui du **drone** sont **confondus**.

Cependant, il est habituel de **commander le drone** en **position** et en **vitesse** dans son propre repère  $R_B$ , et d'utiliser des **matrices de passage** pour exprimer ces grandeurs dans le repère  $R_T$ .

Nous définissons ces matrices de passage comme étant les **matrices d'Euler**, définies ci-dessous :

$$R_\varphi = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & -\sin \varphi \\ 0 & \sin \varphi & \cos \varphi \end{pmatrix}_{R_T} \quad R_\theta = \begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix}_{R_T} \quad R_\psi = \begin{pmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{pmatrix}_{R_T}$$

$$\text{D'où : } \begin{pmatrix} \varphi \\ \theta \\ \psi \end{pmatrix}_{R_B} = R_\varphi \cdot R_\theta \cdot R_\psi \cdot \begin{pmatrix} \varphi \\ \theta \\ \psi \end{pmatrix}_{R_T}$$

Il ne nous reste plus qu'à les implémenter dans notre modèle de simulation afin de pouvoir donner des directives dans le référentiel terrestre qui seront ensuite retranscrites dans le référentiel du drone avant d'être donner au système pour traiter ces directives.

Nous obtenons alors l'architecture logique suivante :

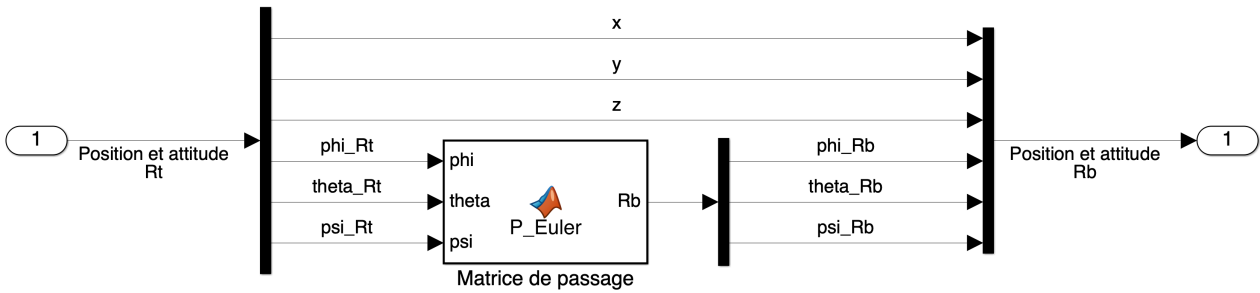


FIGURE 8 : Sous-système Simulink - *Changement de repère Terrestre > Drone.*

### II-2.E Sous-conclusion

Revenons en premier lieu sur les résultats obtenus du TP. Bien que les fichiers de simulation soient accessibles au lecteur, nous proposons les rendus vidéos suivants :

- HOVER SIMPLE :

[Voir la vidéo sur YouTube](#)

- HOVER + ROTATION EN LACET :

[Voir la vidéo sur YouTube](#)

Par ailleurs, voici l'implémentation complète de l'architecture Simulink pour ce premier TP :

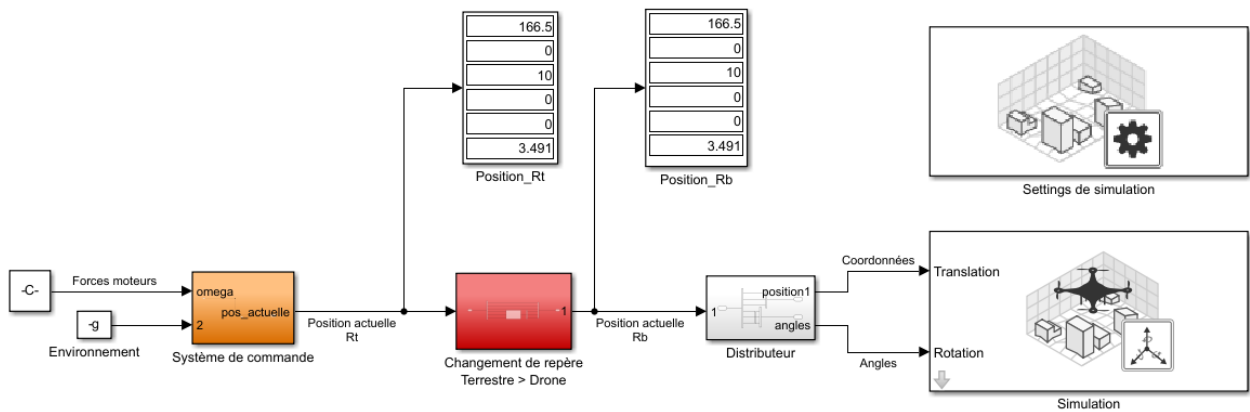


FIGURE 9 : Simulink du contrôle simple du drone - TP\_01

Ce premier TP nous a permis de poser les **bases de la modélisation dynamique** pour un **drone quadrirotor**. Nous avons commencé par mettre en place une **modélisation numérique** du système qui soit le plus **réaliste possible**. Pour cela, le choix d'une **représentation d'état** a été indispensable. Une fois cette structure mise en place, nous avons pu vérifier le bon fonctionnement de celle-ci en créant une commande **"hover"**.

Pendant cette étude, nous avons été confrontés à plusieurs **difficultés** qui ont rendu l'exercice, à première vue simple, **fastidieux**. L'importance de réussir cette première étape était pourtant **essentielle**, car sans une **modélisation correcte**, les prochaines phases du projet auraient été tout simplement **irréalisables**.

➤ Certaines **erreurs**, comme des **incohérences dans le calcul des matrices  $A$** , puis  $B$ , nous ont obligés à **revoir entièrement notre construction dans Simulink** à plusieurs reprises. Ces retours en arrière ont été longs mais **nécessaires** pour garantir un **fonctionnement cohérent** du système.

➤ Nous avons aussi omis un **détail important** lié à l'**intégration avec la UAV Toolbox** : les **angles d'entrée** ne sont pas dans l'ordre  $(\varphi, \theta, \psi)$  comme dans notre modélisation, mais bien dans l'ordre  $(\psi, \theta, \varphi)$ . Cette inversion, bien que sans conséquence dans le TP1, a été **corrigée** pour la suite du projet avec l'ajout d'un **sous-système** simple *Distributeur*.

Cette première partie représente donc une base solide à l'apport de prochaines fonctionnalités à la simulation - l'étude TP\_02.

## II-3 Corps de la simulation - Commande et asservissement (TP\_02)

Nous allons désormais aborder une seconde phase (TP2), plus avancée, qui consiste à commander une **pose complète** du drone. Une pose correspond à une configuration stable dans le repère terrestre, définie par une position  $(x, y, z)$  et une lacet  $\psi$ .

L'objectif est d'amener le drone depuis une pose initiale  $A$ , choisie librement dans la scène 3D, vers une pose finale  $B$ , en assurant le passage d'un équilibre à un autre sans collision avec l'environnement.

Pour cela, nous mettrons en œuvre une commande fermée basée sur le calcul des erreurs entre la pose actuelle et la pose cible : un asservissement. Nous utiliserons des correcteurs PD pour piloter les différentes boucles de correction.

Cette transition marque un changement d'approche : nous ne nous contentons plus de se stabiliser en un unique point, mais nous pilotons activement le système vers une nouvelle position cible, en intégrant les outils de commande nécessaires à ce déplacement contrôlé.

### II-3.A Calculateur des commandes moteurs

La partie **calculateur** intervient en fin de chaîne de commande pour convertir les signaux issus du contrôleur  $(T_c, \varphi_c, \theta_c, \psi_c)$  en vitesses moteurs réelles  $\omega_{M1}, \omega_{M2}, \omega_{M3}, \omega_{M4}$ .

Pour que le drone puisse atteindre une pose cible tout en restant stable en vol, le calculateur traite à la fois les **commandes dynamiques** générées par les correcteurs, et une **contribution constante de poussée**, issue du *compensateur de gravité*. Ce dernier garantit que, même en l'absence de mouvement souhaité, les moteurs produisent une poussée suffisante pour équilibrer le poids du drone.

En combinant ces deux sources d'information, le calculateur permet au drone de se stabiliser naturellement et de réagir efficacement aux ordres de déplacement vers la pose désirée.

Une fois les commandes dynamiques  $(T_c, \varphi_c, \theta_c, \psi_c)$  calculées par les correcteurs à partir des erreurs de pose, il reste à les convertir en quatre commandes moteurs individuelles. Cette conversion est assurée par un bloc appelé **motor mixing**.

Le principe du motor mixing repose sur le fait que chaque moteur n'influence pas uniquement l'altitude, mais également les mouvements de rotation du drone (*Comme vue dans la section 2.E (TP1)*) :

Ainsi, chaque moteur contribue à plusieurs axes de mouvement. Il est donc nécessaire d'utiliser une **matrice de décomposition inverse** pour traduire les commandes globales en efforts moteurs.

La relation matricielle qui permet de faire ce lien est la suivante :

$$\begin{bmatrix} F_{M1} \\ F_{M2} \\ F_{M3} \\ F_{M4} \end{bmatrix} = \frac{M^{-1}}{2} \cdot \begin{bmatrix} T_c \\ \varphi_c \\ \theta_c \\ \psi_c \end{bmatrix}$$

où  $M^{-1}$  est une matrice définie par la géométrie du drone en configuration "X". Dans notre cas, on utilise :

$$M^{-1} = \frac{1}{2} \begin{bmatrix} 1 & -1 & -1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \end{bmatrix}$$

Chaque ligne représente un moteur, et chaque colonne représente une action élémentaire :

- Colonne 1 : altitude  $T_c$  (même signe pour tous les moteurs),
- Colonne 2 : roll  $\varphi_c$  (différence entre côtés),
- Colonne 3 : pitch  $\theta_c$  (différence avant/arrière),
- Colonne 4 : yaw  $\psi_c$  (opposition entre hélices).

Le résultat de ce calcul donne la force individuelle à appliquer à chaque moteur pour que le drone réponde aux commandes globales. Une fois ces forces déterminées, elles seront converties en vitesses de rotation  $\omega_{Mi}$  via la relation :

$$F_{Mi} = k_t \cdot \omega_{Mi}^2 \Rightarrow \omega_{Mi} = \sqrt{\frac{F_{Mi}}{k_t}}$$

En remplaçant  $F_{Mi}$  par son expression :

$$\begin{pmatrix} \omega_{M1} \\ \omega_{M2} \\ \omega_{M3} \\ \omega_{M4} \end{pmatrix} = \sqrt{\frac{M^{-1}}{2 \cdot k_t} \begin{pmatrix} T_c \\ \varphi_c \\ \theta_c \\ \psi_c \end{pmatrix}}$$

Nous pouvons donc déduire  $\omega_{Mi+cg}$  en ajoutant l'expression de la compensation de gravité :

$$\begin{pmatrix} \omega_{M1+cg} \\ \omega_{M2+cg} \\ \omega_{M3+cg} \\ \omega_{M4+cg} \end{pmatrix} = \sqrt{\left[ \frac{M^{-1}}{2 \cdot k_t} \begin{pmatrix} T_c \\ \varphi_c \\ \theta_c \\ \psi_c \end{pmatrix} + \begin{pmatrix} m \cdot g \\ 0 \\ 0 \\ 0 \end{pmatrix} \right]} = \sqrt{\frac{M^{-1}}{2 \cdot k_t} \begin{pmatrix} T_c \\ \varphi_c \\ \theta_c \\ \psi_c \end{pmatrix} + \frac{m \cdot g}{4 \cdot k_t} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}}$$

Nous avons donc bien en sortie du calculateur qui permet au drone de se stabiliser naturellement et de réagir efficacement aux ordres de déplacement vers la pose désirée.

Il est important de noter que les valeurs de  $\omega$  **peuvent être négatives**. Pour pallier à ce problème nous avons appliqué dans la racine une valeur absolue et **conserver le signe** des  $\omega$  en amont de cette dernière de la façon suivante :

```

1 function Rt = Motor_mixing(Vec_F, kt)
2   Rt = sign(Vec_F) .* sqrt(abs(Vec_F) / kt);
3 end

```

Listing 1 : Fonction Motor\_mixing

L'architecture logique du calculateur nous donne donc sur simulink :

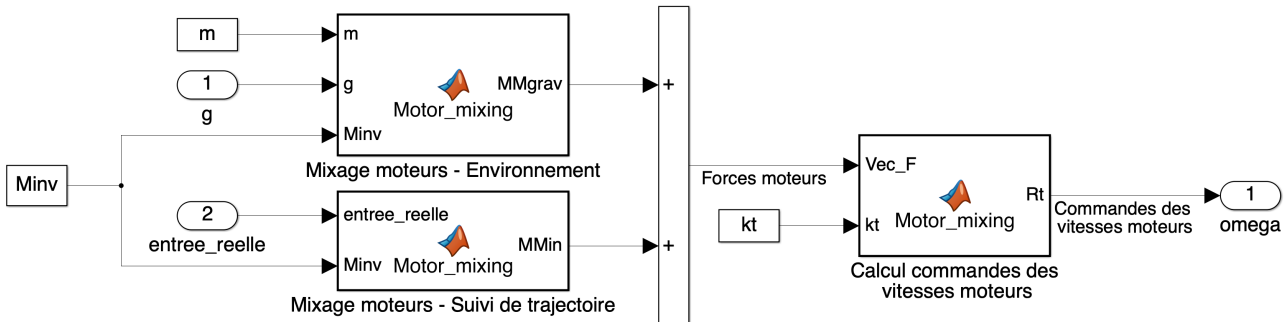


FIGURE 10 : Sous-système Simulink - **Calculateur** : il combine la compensation de gravité et les commandes dynamiques pour générer les forces moteurs, ensuite converties en vitesses de rotation  $\omega_{Mi}$ .

Maintenant que le calculateur est établi, nous le lions à notre bloc de représentation d'état, remplaçant ainsi notre signal initial qui était juste un signal de commande simplifié.

Au lieu d'appliquer directement une poussée constante ou une consigne manuelle de type *hover*, le système reçoit désormais en entrée les **vitesse des moteurs**  $\omega_{Mi}$  calculées dynamiquement à partir des commandes de position et d'orientation  $(T_c, \varphi_c, \theta_c, \psi_c)$ .

Nous obtenons l'architecture logique suivante :

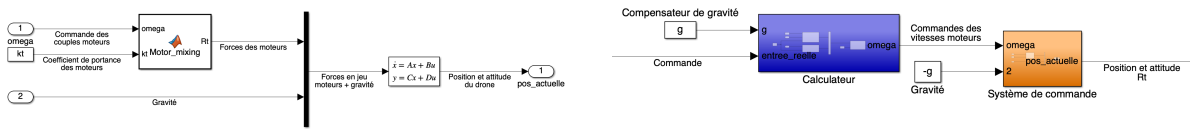


FIGURE 11 : Gauche : Sous-système Simulink - **Système de commande** (TP\_01).  
Droite : Lien *Calculateur* - *Système de commande*

### II-3.B Asservissement du système

Nous cherchons maintenant à **asservir notre système** afin de pouvoir **contrôler notre drone** de manière à ce qu'il **obéisse à une consigne** que nous lui donnerons. Nous cherchons, dans le cadre de ce projet, à ce que notre drone soit à la fois **réactif** tout en maintenant une **réponse stable**. Nous avons donc choisi d'utiliser des **correcteurs PD**. Les **valeurs** de ce PD ont été déterminées par l'un de nos collaborateurs, **Monsieur J.** Nous avons retenu un **proportionnel** de 0.2 et un **dérivé** de 0.7.

Afin d'**asservir notre système**, nous commençons par ajouter des **capteurs** dans notre schéma Simulink, ici représentés par le **bloc vert**, que l'on peut voir relié **directement à la sortie du système**. Ce bloc a pour rôle de **récupérer les valeurs de position et d'angles** du drone, nécessaires pour calculer les erreurs d'asservissement.

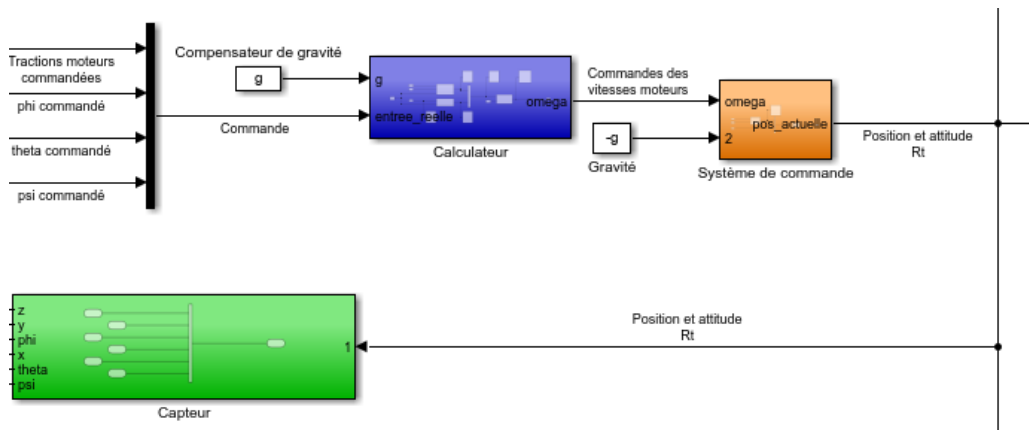


FIGURE 12 : Récupération des données

Nous considérons le **capteur comme parfait**, autrement dit, nous imposons un **retour unitaire** sur chacune des variables. Le choix de ne récupérer que les **valeurs de position et d'angle** est un **choix fort**, car nous perdons ainsi une part importante des **capacités de contrôle** liées à la **vitesse** ou à l'**accélération**. Nous faisons ce choix en pleine connaissance de cause, afin de **simplifier notre étude du système** et laisser notre **asservissement** gérer automatiquement les autres variations.

Une fois les **informations récupérées par le capteur**, il nous faut les **soustraire aux valeurs de consigne** pour obtenir une **erreur**. Nous effectuerons cette étape en **deux temps** :

- D'abord, nous créons une **erreur en  $x$  et  $y$** , qui sera **retranscrite en consignes de pitch ( $\theta$ ) et de roll ( $\varphi$ )** ;
- Ensuite, ces consignes sont comparées, en parallèle avec les **valeurs d'altitude ( $z$ ) et de yaw ( $\psi$ )**, pour obtenir une **asservissement complet**.

Vous pouvez voir ci-dessus le **schéma de l'asservissement** tel qu'il est implémenté dans notre modèle Simulink.

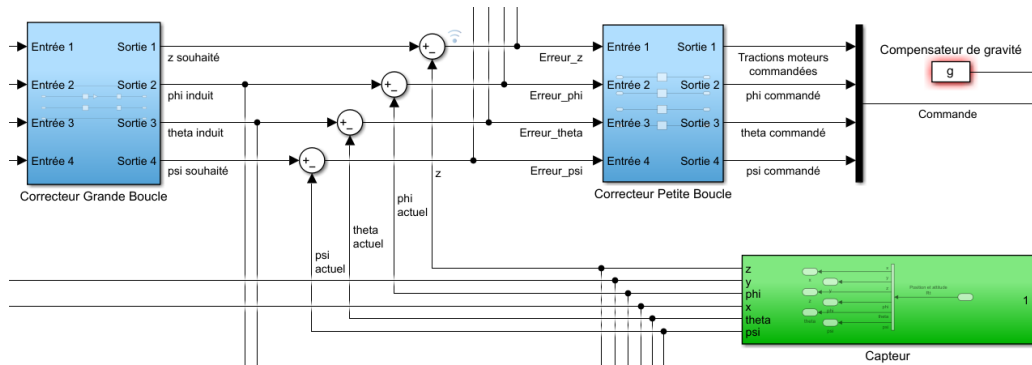
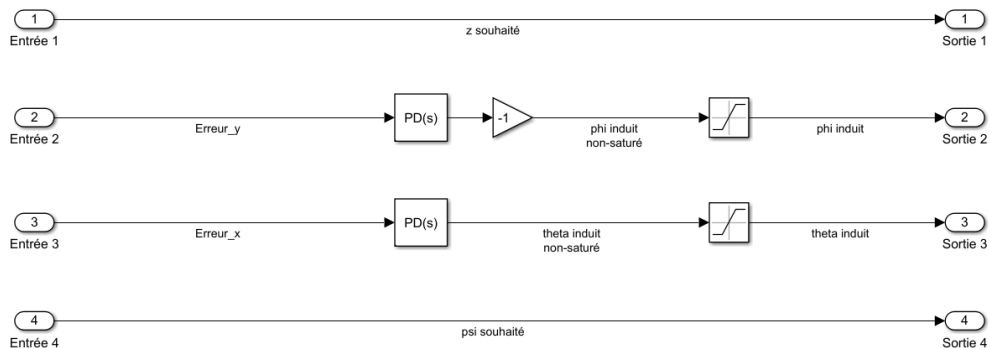


FIGURE 13 : Asservissement

Commençons par le **premier bloc bleu ciel**. Il s'agit du **bloc d'asservissement** de ce que nous appelons la **grande boucle**. Comme expliqué précédemment, dans ce bloc nous **comparons les valeurs réelles et souhaitées** de  $x$  et  $y$  afin d'en déduire des **valeurs de rotation en roll ( $\varphi$ ) et pitch ( $\theta$ )**.

L'intérieur du bloc est donc composé de la façon suivante :

FIGURE 14 : Sous-système Simulink - **Correcteur Grande Boucle**

Il est important de noter que **tous les correcteurs PD** utilisés ici ont les **mêmes propriétés**. De plus, vous aurez sûrement remarqué la présence d'un **bloc de gain inversant le signe** de notre **erreur en  $y$** . En effet, nous comptons l'ensemble des **angles comme positifs dans le sens trigonométrique**. Cependant, si nous voulons nous déplacer selon la direction  $y$ , il nous faut **pivoter dans le sens horaire**, d'où la **nécessité d'une inversion de signe**.

Enfin, il nous faut aussi **ajouter des saturations**. En effet, une **forte erreur** pousse le drone à la **corriger rapidement**, ce qui peut se traduire par un **angle de rotation élevé**. Cela peut remettre en cause l'**hypothèse des petits angles**, essentielle dans notre modélisation. Nous imposons donc une **saturation à  $\pm 0.25$  radian** pour garantir une cohérence avec les hypothèses physiques du modèle.

Nous effectuons les mêmes opérations sur le correcteur petite boucle en ajoutant des PDs sur chaque variable.

### II-3.C Sous-conclusion

Ce second TP constitue une avancée majeure dans le contrôle du drone, puisqu'il ne s'agit plus seulement de stabilisation en vol stationnaire, mais bien de permettre au drone d'atteindre une **pose cible complète**  $(x_d, y_d, z_d, \psi_d)$  à partir d'une pose initiale  $(x, y, z, \psi)$ . Pour cela, nous avons mis en place une architecture de commande fermée, fondée sur le calcul d'erreurs de pose et la régulation dynamique de la trajectoire.

➤ La mise en œuvre des **correcteurs PD** sur les quatre grandeurs  $(x, y, z, \psi)$  nous a permis de générer les consignes de commande intermédiaires  $(T_c, \varphi_c, \theta_c, \psi_c)$ , nécessaires au pilotage du drone dans son environnement.

➤ Ces commandes ont ensuite été traitées par un **calculateur**, composé d'un bloc de **compensation de gravité** et d'un **motor mixing**, assurant une réponse adaptée aux commandes.

Nous pouvons alors présenter le résultat final du TP, qu'est la réalisation du parcours urbain choisi par le drone. Amateurs de sensations fortes, vous serez servis ([lunettes de vitesse conseillées](#)).

1. ➤ PARCOURS :

[Voir la vidéo sur YouTube](#)

L'architecture logique finale Simulink est présentée en annexe.

En ce qui concerne les erreurs et **difficultés rencontrées** lors de cette étude, nous pouvons en premier lieu citer la **mise en place de l'asservissement**. En effet, le fait de séparer chaque signal d'état ne nous avait pas paru évident à nos débuts (bien qu'essentielle).

En dehors de ceci, la difficulté principale fut la **résolution d'un problème de réponse du drone** en position B lorsque nous lui commandions une position C dans le même demi-espace que la position A. (*Exemple : bas - haut - bas*).

En effet, une racine carrée de la valeur absolue des forces motrices pouvait (avant correction) ne **pas prendre en compte un potentiel signe** - de celles-ci. Nous avons donc corrigé cette erreur dans le bloc matlab "*Calcul commandes des vitesses moteurs*" avec le code présenté plus tôt.

Ce TP2 clôt ainsi la structure fondamentale de notre contrôleur de vol, en introduisant pour la première fois une logique d'**asservissement**, que nous pourrions réutiliser et améliorer dans les extensions futures des demandes du client.

## II-4 Annexe

L'image de notre système au complet est trop grande pour être affichée sur notre pdf nous vous proposons donc un lien pour pouvoir l'obtenir. Nous vous conseillons de la **télécharger** pour un meilleur rendu.

Cliquez sur le lien ci-dessous pour voir l'image en PDF :

[Voir l'image PDF en haute qualité](#)

## II-5 Propositions supplémentaires

Une fois notre modélisation et simulation implémentées sous Matlab-Simulink, nous avons fais le choix d'ajouter certaines fonctionnalités à l'ensemble. Ces ajouts s'inscrivent dans l'objectif d'une meilleure prise en main et compréhension de la simulation.

Entamons cette section avec la première addition : un suivi de trajectoire à plus de deux positions.

### II-5.A Suivi de position

Disséquons le fonctionnement logique souhaitable d'un tel système :

1. ➤ **ACCÈS À LA POSITION ACTUELLE** : Du fait du **capteur implémenté dans le TP\_02**, cette étape n'est pas des plus compliquées. Nous ajoutons simplement un **retour de la position actuelle** au sous-système "*Suivi de position*".
2. ➤ **ACCÈS À LA POSITION SUIVANTE** : Pour cette étape, il apparaît que l'**utilisateur doit à présent fournir une liste de  $N$  positions**, et non plus une **unique position d'arrivée désirée**.

Notre drone doit en effet passer d'un **parcours simple** :

$$Pos_1 \rightarrow Pos_2$$

à un **parcours complet** :

$$Pos_1 \rightarrow \dots \rightarrow Pos_i \rightarrow \dots \rightarrow Pos_N$$

Nous définissons pour ceci la liste matricielle *entree\_reelle* de taille  $4 \times 1 \times N$  :

$$\begin{array}{c} \text{entree\_reelle}(:, :, 1) \\ \left[ \begin{array}{c} z_1 \\ y_1 \\ x_1 \\ \varphi_1 \end{array} \right] \end{array} \quad \begin{array}{c} \text{entree\_reelle}(:, :, N) \\ \left[ \begin{array}{c} z_N \\ y_N \\ x_N \\ \varphi_N \end{array} \right] \end{array}$$

FIGURE 15 : Liste matricielle à  $N$  dimensions des positions désirées.

3. ➤ **CALCUL D'ERREUR ET CONDITION DE VALIDITÉ** : Avec la connaissance de la **position actuelle du drone** ainsi que de toutes celles qu'il doit suivre, il est possible, à l'aide du compteur *CurrentIndex*, de **comparer ces deux données**. Nous en tirons ainsi une **erreur en position** ainsi qu'une **erreur en lacet**.
4. ➤ **CHANGEMENT DE POSITION** : Finalement, en ajoutant une **condition d'inégalité** entre nos **erreurs** et des **tolérances** définies en amont, nous disposons d'un moyen pour que le drone **valide l'arrivée à une position cible**. En définissant alors  $NewIndex = CurrentIndex + 1$ , **nouvel indice de position** dans *entree\_reelle*, notre drone est capable de **changer de position cible automatiquement** !

Nous ajoutons également un autre système logique de fin de simulation en cas de dépassement par *NewIndex* de *N*. Nous obtenons alors l'architecture logique suivante :

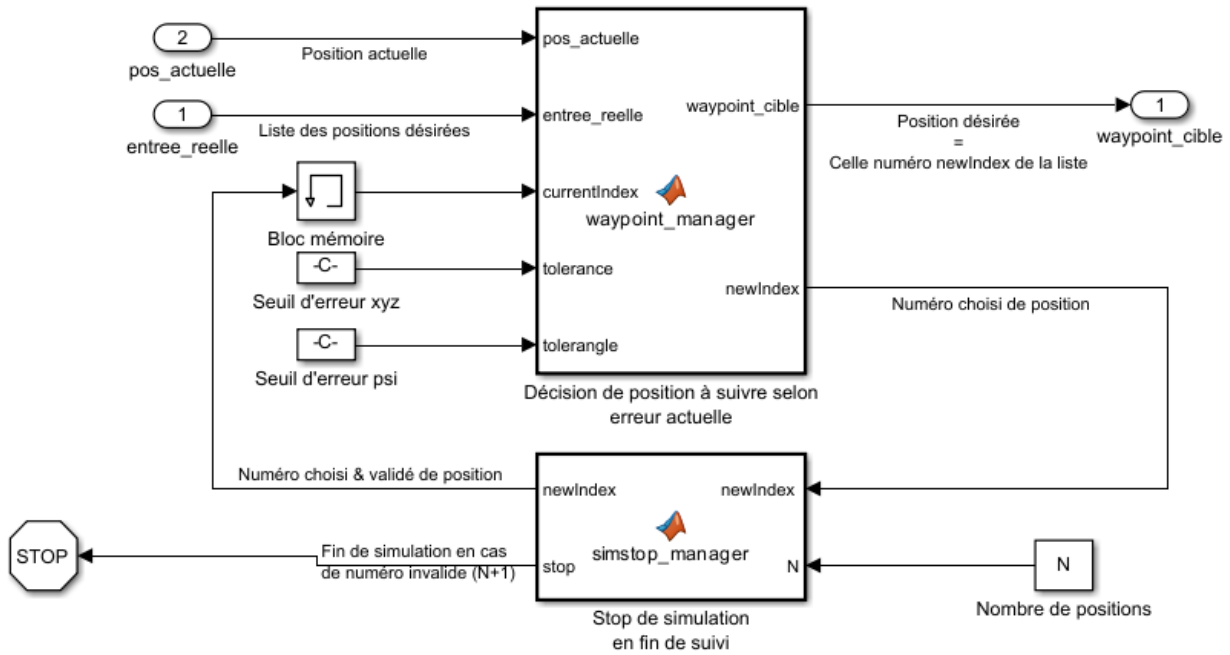


FIGURE 16 : Sous-système Simulink - *Suivi de trajectoire et arrêt de simulation.*

## II-5.B Parcours urbain

L'algorithme de suivi de position étant au point, nous pouvons à présent le mettre en oeuvre en proposant le parcours suivant :

```
% Liste de positions à suivre :
%
entree_reelle(:,1) = [0; 0; 0; deg2rad(0)];
entree_reelle(:,2) = [8; 0; 0; deg2rad(0)];
entree_reelle(:,3) = [8; -2; 118; deg2rad(0)];
entree_reelle(:,4) = [2; -15; 125; deg2rad(-70)];
entree_reelle(:,5) = [1; -23; 131; deg2rad(-55)];
entree_reelle(:,6) = [1; -25; 145; deg2rad(-28)];
entree_reelle(:,7) = [0.8; -32; 155; deg2rad(-45)];
entree_reelle(:,8) = [3; 50; 160; deg2rad(90)];
entree_reelle(:,9) = [8; 60; 160; deg2rad(135)];
entree_reelle(:,10) = [27; 60; 120; deg2rad(90)];
entree_reelle(:,11) = [27; 105; 120; deg2rad(-90)];
entree_reelle(:,12) = [5; 105; 40; deg2rad(-160)];
entree_reelle(:,13) = [5; 48; 20; deg2rad(-105)];
entree_reelle(:,14) = [26; 48; 20; deg2rad(-90)];
entree_reelle(:,15) = [38; -3; 20; deg2rad(-90)];
entree_reelle(:,16) = [51; -16; 22; deg2rad(-78)];
entree_reelle(:,17) = [45.8; -16; 22; deg2rad(0)];
```

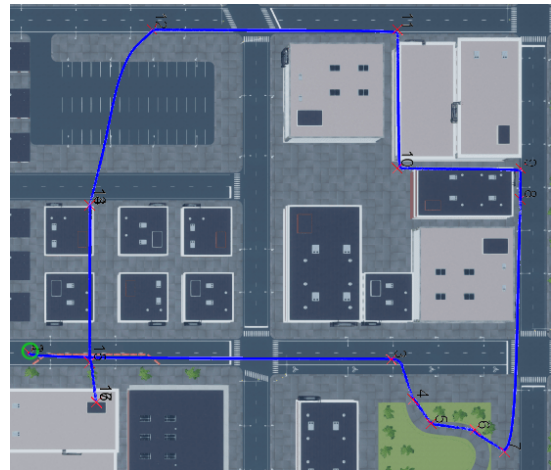


FIGURE 17 : Listes des positions (gauche) composant le parcours (droite) du drone.

Démarrant du **point 1** pour finir sur le **toit d'un bâtiment** au **point 17**, ce **parcours** nous fournit de **riches informations** sur le comportement du drone. Il est en effet possible, lorsqu'on **visualise la simulation**, de tirer certaines **conclusions** sur les **capacités actuelles** de notre quadrimoteur.

- **CAPACITÉ VARIÉE DE SUIVI** : En premier lieu, nous pouvons valider la **capacité du drone à répondre correctement** à des **demandes en position proches** les unes

des autres ( $Pos_4 \rightarrow Pos_5$ ) ou au contraire **très espacées** ( $Pos_{12} \rightarrow Pos_{13}$ ).

2. ➤ DÉPASSEMENTS INERTIELS : En second lieu, nous remarquons la **fâcheuse habitude** de notre drone à **dépasser à plusieurs reprises ses consignes d'altitude** (positivement ou négativement). Des **rendus graphiques** sur l'**altitude en 2D** et la **trajectoire 3D** permettent de constater rigoureusement ce phénomène.

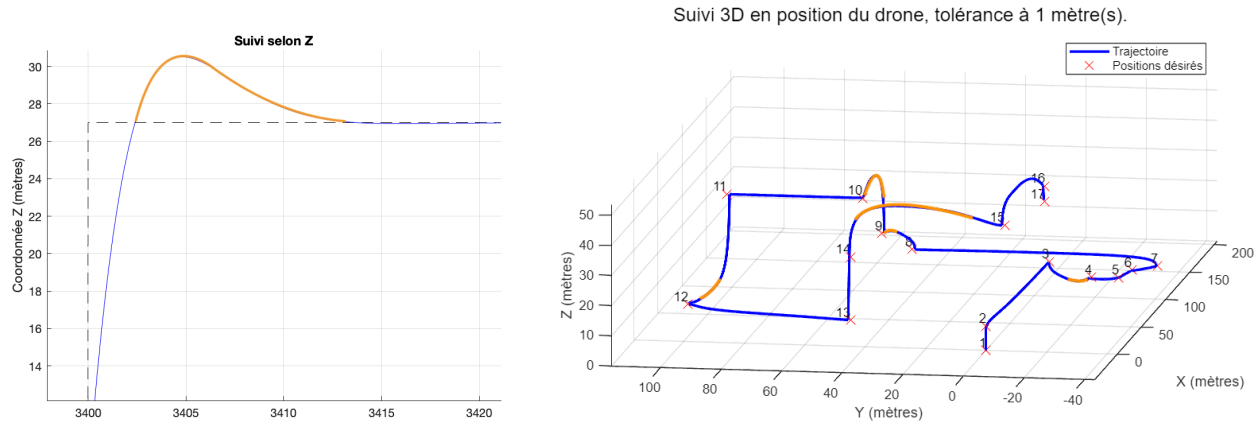


FIGURE 18 : Focus sur le passage au point 10 (gauche) et trajectoire complète du drone (droite). En orange apparaissent les dépassements.

### III OUVERTURE

Comme nous avons pu le constater, le **pilotage autonome** d'un drone dans un **environnement simulé** comporte bien plus de **complications** que l'on ne le croit. Il s'agit de concevoir un **système** capable de comprendre sa propre **position**, d'en déduire les **écarts** face à une **cible**, et d'agir de manière **logique** et **stable** pour corriger ces écarts en **temps réel**.

Ce projet rentre dans cet objectif : partir d'une **modélisation physique**, poser une **représentation d'état** juste, aboutir à un **enchaînement de commande**, **hiérarchisée** et **adaptable**.

À travers les deux premières parties (TP1 et TP2), nous avons progressivement construit un **système** permettant au drone de passer d'un simple **maintien en vol**, à un **déplacement contrôlé**, intégrant des **boucles d'asservissement**, un **calculateur** et une **conversion des commandes en efforts moteurs**.

Ce travail marque ainsi une **étape fondamentale** dans notre **compréhension** et **mise en œuvre** de **systèmes de contrôle** dans l'**aéronautique**. Les fondations posées ici, bien que très allégées, constituent les **bases du vol autonome**.

Dans le cadre où l'on voudra développer le projet vers des cas d'usage plus réalistes, plusieurs pistes peuvent être envisagées.

- ➤ Nous pourrions **casser certaines hypothèses** de simplification utilisées dans notre modèle actuel (petit angles, absence de traînée, constance de la gravité, ect) afin d'approcher un comportement plus réaliste du drone.
- ➤ **L'introduction de correcteurs PID** sur chaque boucle d'asservissement en vue d'améliorer **la stabilité et la réactivité**, notamment en présence de perturbations.
- ➤ La possibilité d'intégrer des capteurs physiques (IMU, baromètre, GPS, ect) avec des données avec la simulation de présence de bruits.
- ➤ **Brancher un manette** directement à l'ordi pour prendre le contrôle du drone à la main - que ce soit pour tester le système dans ces conditions plus crédibles.
- ➤ **Remplacer la représentation par angles d'Euler** par une méthode plus robuste, pour éviter des problèmes connus comme le **Blocage de cadran** (*plus communément appelé gimball lock*), qui limite fortement la liberté de mouvement et complique le calcul des changements de repère en 3D.

Il y a donc plein de façons de faire évoluer ce que l'on a construit ici et aller plus loin dans la simulation ou le contrôle embarqué.

*Nous remercions notre professeur d'automatique pour nous avoir guidés à travers ce projet, mais aussi tout au long de l'année. Il fait partie de ces enseignants qui souhaitent réellement nous offrir une formation sérieuse en vue de notre avenir. Sa motivation et sa bienveillance ont été grandement appréciées.*